WATT-WATCH — Person 2 guide: Scanning & Prediction

> **You are Person 2.** You are responsible for the **meter scanning (OCR)**, the **bill prediction** and the **early warning** — testing them, explaining them in the report, and presenting them.
>
> You **do not need to install or run the code** — Person 1 runs the app. (If you want to run it yourself, see the Appendix at the end.)
>
> **Deadline: 2 days.** Follow the steps in order.

1. What is WATT-WATCH? (read this first)

In Tamil Nadu, electricity is billed in **slabs**: the more units you use in a billing cycle (about 2 months), the higher the price of each extra unit. Families usually find out how much they used only when the bill arrives.

WATT-WATCH is a **web application** (it runs in a browser) that:

```
Scan meter → Track units → Predict bill → Warn before a costlier slab → Reduce usage → See savings
```

Your part is the first four steps: Scan, Track, Predict, Warn.

The team:

Person	Role
Person 1 (team lead)	Runs the app · database · tariff & bills · final report
You (Person 2)	Meter photos (OCR) · prediction · early warning
Person 3	Appliances · saving plan · slides · demo video

2. Your job at a glance

When	Task	Time	Hand in
Day 1 morning	Step A: Get the code & join the team meeting	1 h	—
Day 1 afternoon	Step B: Understand your part	1 h	Your notes

When	Task	Time	Hand in
Day 1 afternoon	Step C: Take meter photos	1 h	10 photos + real readings → Person 1
Day 1 afternoon	Step D: Your home's readings	15 min	Readings → Person 1
Day 1 evening	Step E: OCR accuracy table	30 min	Accuracy table
Day 1 evening	Step F: Write report section 7.2	1–2 h	Section 7.2
Day 2	Step G: Results & limitations, practise	full day	Sections 8 & 9

Step A — Get the code and join the team meeting (Day 1 morning)

A1. Download the code (5 minutes)

1. Open **https://github.com/ZumairaSultana/WATTWATCH** in your browser.
2. Near the top left there's a button that says **main** — click it and choose **claude/code-implementation-guide-fqx9ks**. *(The code is on this branch, not on main.)*
3. Click the green **Code** button → **Download ZIP**.
4. Open your **Downloads** folder → right-click the ZIP → **Extract All** → **Extract**. You get a folder whose name starts with **WATTWATCH-**.
5. *(Optional)* Install **VS Code** (code.visualstudio.com) to read the code with colours: **File → Open Folder** → pick that folder.

A2. Team meeting (1 hour)

- Person 1 shows the app. Watch especially the 📷 **Scan & Track** and 📈 **Predict & Warn** tabs.
- Person 1 shares the **WATT-WATCH Report** Google Doc with you — open it and find **7.2 Scanning & prediction**.

Step B — Understand your part (1 hour)

Your files are in the `wattwatch` folder:

File	What it does
`ocr.py`	Reads the number from a meter photo
`tracker.py`	Keeps the readings of the current cycle and calculates units used
`predictor.py`	Predicts the units and bill at the end of the cycle
`alerts.py`	Warns before usage crosses into a costlier slab

B1. Scanning — OCR (`ocr.py`)

OCR (Optical Character Recognition) = the computer reading text or numbers from a picture. We use **Tesseract**, a free OCR program.

What happens when the user uploads a meter photo:

Step	What the code does	Why
1	Turns the photo the right way up	Phone photos are often stored sideways
2	Makes it **grey** (no colour)	Colour doesn't help read digits
3	Makes it **bigger** (1000 pixels wide)	Tesseract reads larger text better
4	**Stretches the contrast**	Digits stand out from the background
5	**Removes speckle noise** (median filter)	Small dots aren't mistaken for digits
6	Turns every pixel **pure black or white**	Cleanest input for OCR
7	Tesseract reads it, **allowed only digits and "."**	Prevents letters being "read"
8	Tries again with colours **inverted**	Some displays are light-on-dark
9	Picks the **longest number** found	The reading is usually the biggest number on the display

The code for steps 1–6:

```
image = ImageOps.exif_transpose(image)          # 1 rotate correctly
gray = ImageOps.grayscale(image)                # 2 grey
if gray.width < 1000:                           # 3 enlarge
    scale = 1000 / gray.width
    gray = gray.resize((1000, int(gray.height * scale)))
gray = ImageOps.autocontrast(gray)              # 4 contrast
gray = gray.filter(ImageFilter.MedianFilter(3)) # 5 remove noise
return gray.point(lambda p: 255 if p > 128 else 0)  # 6 black or white
```

Important design choice: OCR can be wrong (glare, angle, blurry photo), so the app **shows the number in a box and the user checks/corrects it before saving.**

B2. Tracking (`tracker.py`)

- A **billing cycle** starts at the reading printed on the last bill.
- Each new reading is saved with its date.
- **Units used so far = latest reading − starting reading.**

Example: last bill reading **5000** on 1 Oct; today (18 Oct) the meter shows **5080** → **80 units** used in **17 days**.

B3. Prediction (`predictor.py`)

Idea: if you plot *meter reading* against *day*, it's close to a **straight line** for a family with a normal routine. The **slope** of that line = **units per day**.

- With **2 readings**: units per day = (difference in reading) ÷ (difference in days).
- With **3 or more readings**: the code fits the **best straight line** through the last 5 readings (**least squares**, `numpy.polyfit`). This smooths out one unusual day.

```
if len(points) == 2:
    slope = (y[1] - y[0]) / (x[1] - x[0])
else:
    slope = np.polyfit(x, y, 1)[0]   # slope of the best-fit straight line
```

Then:

```
predicted units = units so far + units per day × days left in the cycle
predicted bill  = bill calculated for the predicted units
```

Worked example (60-day cycle):

	Value
Units so far	80 (in 17 days)
Units per day	80 ÷ 17 ≈ **4.7**
Days left	60 − 17 = **43**
Predicted units	80 + 4.7 × 43 ≈ **282**
Predicted bill	(282 − 100 free) × Rs 4.80 ≈ **Rs 874**

B4. Early warning (`alerts.py`)

The slab limits are **100, 400, 500, 600, 800, 1000** units. The warning checks which limits lie **between "units so far" and "predicted units"**:

```
crossing = [b for b in tariff.boundaries if p.units_so_far < b < p.projected_units]
```

- **No limit in between** → ✅ **Safe** (green): "On track..."
- **One or more limits in between** → the app calculates:
  - **days until the next limit** = (limit − units so far) ÷ units per day
  - **target** = the nearest limit *below* the prediction
  - **units to cut** = predicted units − target
  - **money at stake** = bill(predicted) − bill(target)
- **Level:**
  - 🔴 **Critical** (red) if the next limit is ≤ 7 days away **or** 2+ limits will be crossed
  - 🟠 **Warning** (orange) otherwise

In the example above: 80 units so far, 282 predicted → the limit **100** is in between → crossed in (100 − 80) ÷ 4.7 ≈ **4 days** → **Critical**.

> ⚠️ **Known limitation — be ready to explain it:** because the first 100 units are free, almost every home crosses 100 early, so the app often says "cut about X units to stay at or below 100", which isn't realistic. Say: *"The warning always aims for the nearest slab limit below the prediction. A better version would ignore the free-units limit or let the user choose a target. We list this under future work."*

Step C — Take meter photos (1 hour)

Goal: measure how well the OCR reads real meters.

1. Take **10 photos** of electricity meter displays — your home, neighbours, relatives (**ask permission**).
2. Mix the conditions so the test is fair:
   - **4 clear** photos (good light, straight on, close)
   - **3 dim light** photos (evening, shadow)
   - **3 at an angle** (from the side)
3. For each photo, **write down the real reading** you can see with your eyes. Name the photos `photo1`, `photo2`, … so they match your list.
4. Send the photos + your list to **Person 1** (WhatsApp as *document*, or Google Drive — sending as a document keeps full quality).

Tips: avoid flash glare on the glass; fill most of the photo with the display; keep the phone steady.

Step D — Your home's readings (15 minutes)

Send Person 1:

1. Your **meter number** (printed on the meter or the bill).
2. The **reading and date on your last electricity bill**.
3. **Today's meter reading** (and today's date).

Person 1 adds your home to the app and sends you a screenshot of 📈 **Predict & Warn**. From it, write down: units so far · units per day · predicted units · predicted bill · warning message and level.

Step E — OCR accuracy table (30 minutes)

Person 1 tests each photo in the app and tells you what the app read. Fill in:

Photo	Real reading	App read	Correct?	Condition (clear / dim / angle)
photo1				
photo2				
…				

Accuracy = number correct ÷ 10 × 100. Example: 6 correct → 6 ÷ 10 × 100 = **60%**.

Also count accuracy **per condition** (e.g. clear 4/4, dim 1/3, angle 1/3) — this shows *why* it fails.

> A low accuracy is **fine** if you explain it honestly. It's the reason the app lets the user check and correct the number before saving.

Step F — Write report section 7.2 (evening, 1–2 hours)

Open the Google Doc → **7.2 Scanning & prediction**. Write in your own words:

7.2.1 Meter scanning (OCR) — what OCR is; Tesseract; the 9 steps (table in B1); why the user confirms the number.

7.2.2 Tracking usage — cycle starts at the last bill reading; units so far = latest − start (example from B2).

7.2.3 Bill prediction — the straight-line idea; units per day (2 readings vs least squares); the formula; the worked example table (B3).

7.2.4 Early warning — slab limits; how "crossing" is detected; days until crossing; units to cut; safe/warning/critical levels.

7.2.5 Results — your OCR accuracy table and percentage; your home's prediction screenshot and numbers.

Step G — Day 2

Time	You
9:00 – 11:00	Write **8 Results** (collect everyone's tables: Person 1's bill check, your OCR accuracy & predictions, Person 3's saving plan) and **9 Limitations** (list below).
11:00 – 13:00	With the team: read the whole report, fix mistakes.
15:00 – 17:00	Practise the full presentation **3 times**.
17:00 – 18:00	Practise the questions (Section 4 below).

Limitations to include in section 9:

- OCR struggles with glare, angles, dim light and segmented (7-segment) digits.
- Prediction assumes daily usage stays steady; festivals, guests or weather changes affect it.
- The early warning often targets the 100-unit free limit, which isn't realistic.
- Real bills for our homes had not arrived yet, so the full-cycle prediction couldn't be compared with an actual bill within the project time.

- Tariff rates must be updated when the government revises them.
- Meter roll-over (e.g. 99999 → 00000) is not handled.

---

3. Your part of the presentation (slides 7–8)

Slide	What to show	What to say (in your own words)
7. Scanning (OCR) & accuracy	The 9-step table (short version) and your accuracy result	"The user uploads a meter photo. We clean the image — grey, bigger, black and white — and Tesseract reads only digits. We tested 10 real photos and got X% correct; clear photos worked best. Because OCR can be wrong, the user always confirms the number."
8. Prediction & early warning	The worked example table and a Predict & Warn screenshot	"From the readings we calculate units per day as the slope of a straight line. Units so far plus units per day times days left gives the predicted units and bill. If the prediction crosses a slab limit, the app warns early — how many days until we cross and how many units to cut."

4. Questions you may be asked

Question	Answer
What is OCR?	Optical Character Recognition — the computer reading text/numbers from an image. We use Tesseract.
Why convert to black and white?	It removes colours and shadows so the digits stand out — Tesseract reads clean images better.
What if OCR reads the wrong number?	The user sees the number in a box and corrects it before saving.
How does the prediction work?	Units per day = slope of a straight line through recent readings; predicted = units so far + units per day × days left.
What is least squares / polyfit?	A method to find the straight line that fits several points best, so one unusual day doesn't distort it.
Why only the last 5 readings?	So the prediction follows recent changes, e.g. more AC use in summer.
When is a warning "critical"?	When the next slab limit is 7 days away or less, or more than one limit will be crossed.
Why does it say cut to 100 units?	It targets the nearest slab limit below the prediction; the free-100 limit makes this unrealistic — future work.
Is it an app or a website?	A web application — runs in any browser, built with Python and Streamlit.

5. If something goes wrong

Problem	Fix
Can't find the branch on GitHub	Click the button that says **main** near the top left → type `claude` → pick the branch
ZIP won't open	Right-click → **Extract All** first; don't open files inside the ZIP directly
Photos too big for WhatsApp	Send via Google Drive and share the link with Person 1
Don't understand a part	Ask Person 1 to show it in the running app, then re-read Step B

Appendix — Run the app on your own laptop (optional, ~45 minutes)

Only if you want to try it yourself (useful for the viva, not required).

1. Install **Python** from python.org → ⚠️ tick **"Add python.exe to PATH"** → **Install Now**.
2. Open the extracted **WATTWATCH-...** folder in File Explorer → click the address bar → type **cmd** → **Enter**. *(This opens Command Prompt already inside the folder.)*
3. Type one line at a time:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python -m pytest
streamlit run app.py
```

4. ✅ Tests show `32 passed`; the browser opens the app at **http://localhost:8501**.
5. To stop: click the Command Prompt window → **Ctrl + C**.
6. Reading photos also needs **Tesseract** (search "Tesseract UB Mannheim"); without it, type readings by hand.